BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN HỌC

ĐỀ TÀI: REST API QUẢN LÝ SINH VIÊN VỚI API GATEWAY, LAMBDA VÀ DYNAMODB

Giảng viên hướng dẫn: Th.S Huỳnh Xuân Phụng

Tên học phần: Điện toán đám mây

Mã lớp học phần: CLCO332779_05UTExMC

Nhóm sinh viên thực hiện:

STT	Họ và tên	MSSV
1	Phan Nguyễn Quốc Hoàng	24110219
2	Phạm Trần Đức Lương	24110281

Tp. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

1. Giới thiệu đề tài 1
1.1. Vấn đề 1
1.2. Mục tiêu 1
1.3. Phạm vi dự án 1
2. Kiến trúc hệ thống 2
2.1. Sơ đồ và luồng hoạt động 2
2.2. Dịch vụ được lựa chọn và lý do 2
2.3. Ràng buộc của AWS Learner Lab 3
3. Triển khai hệ thống 4
3.1. Khởi tạo cơ sở dữ liệu và lưu trữ giao diện 4
3.2. Xây dựng AWS Lambda 5
3.3. Tạo REST API bằng Amazon API Gateway 5
3.4. Tích hợp frontend với API 6
3.5. Thiết lập giám sát và kiểm thử 7
4. Bảo mật 8
4.1. Quản lý quyền truy cập bằng IAM 8
4.2. Bảo mật mạng và kiểm soát truy cập 9
4.3. Bảo vệ API Key và thông tin nhạy cảm 9
4.4. Tổng kết 9
5. Kiểm thử và giám sát hệ thống 9
5.1. Kiểm thử chức năng API 10
5.2. Kiểm thử tích hợp End-to-End 11
5.3. Giám sát hệ thống 13
6. Chi phí và tối ưu hóa 15

6.1. Chi phí ghi nhận 15
6.2. Các biện pháp tối ưu chi phí 16
6.3. Quản lý và dọn dẹp tài nguyên 17
7. Những vấn đề gặp phải và bài học kinh nghiệm 18
7.1. Các vấn đề gặp phải 18
7.2. Bài học kinh nghiệm 19
8. Đóng góp của các thành viên và khai báo sử dụng AI 19
8.1. Đóng góp của các thành viên 19
8.2. Khai báo sử dụng AI 23
TÀI LIỆU THAM KHẢO 25

1. Giới thiệu đề tài

1.1. Vấn đề

Việc xây dựng và vận hành các hệ thống quản lý sinh viên theo phương pháp truyền thống thường yêu cầu duy trì máy chủ vật lý hoặc máy chủ ảo. Điều này không chỉ phát sinh chi phí duy trì tài nguyên hạ tầng mà còn cần quản trị, cấu hình và bảo trì máy chủ trong suốt quá trình vận hành. Bên cạnh đó, việc mở rộng tài nguyên khi lưu lượng truy cập tăng cũng cần được tính toán và cấu hình phù hợp.

Xuất phát từ những hạn chế trên, dự án lựa chọn mô hình điện toán không máy chủ trên nền tảng Amazon Web Services, cho phép tận dụng các dịch vụ có sẵn để triển khai ứng dụng mà không cần trực tiếp quản lý máy chủ.

1.2. Mục tiêu

Dự án hướng đến nghiên cứu, xây dựng và triển khai một hệ thống REST API quản lý sinh viên sử dụng kiến trúc Serverless trên nền tảng AWS. Các mục tiêu cụ thể bao gồm:

- Xây dựng các chức năng thêm mới, truy vấn, cập nhật và xóa thông tin sinh viên thông qua REST API.
- Tích hợp các dịch vụ Amazon S3, Amazon API Gateway, AWS Lambda và Amazon DynamoDB để xây dựng hệ thống hoàn chỉnh từ giao diện người dùng đến cơ sở dữ liệu.
- Sử dụng Amazon DynamoDB để lưu trữ thông tin sinh viên, kết hợp Global Secondary Index (GSI) nhằm hỗ trợ truy vấn dữ liệu theo chuyên ngành.
- Kết hợp API Key và Usage Plan để quản lý việc truy cập và giới hạn lưu lượng yêu cầu đến API.
- Sử dụng Amazon CloudWatch để thu thập nhật ký, theo dõi số lượng yêu cầu và độ trễ của API.

1.3. Phạm vi dự án

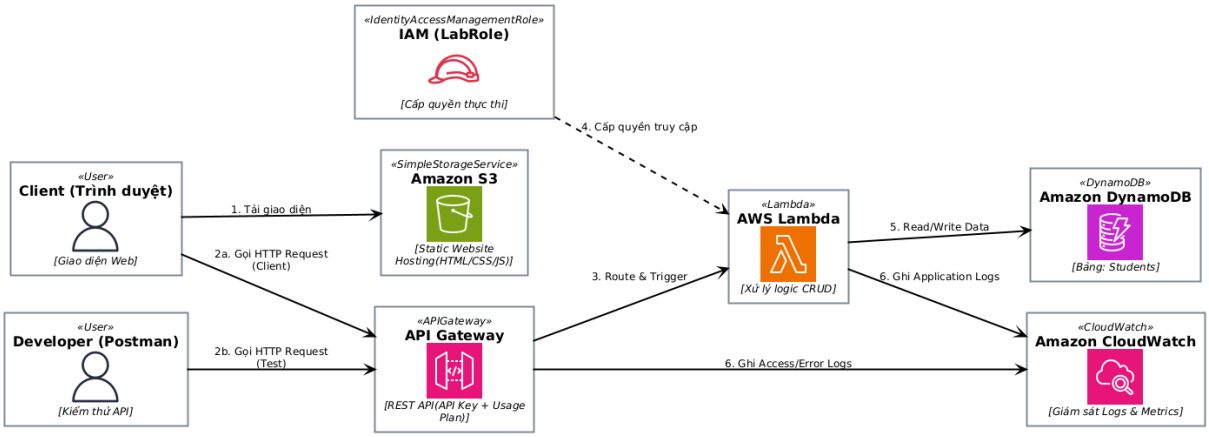
Mô hình Serverless REST API quản lý sinh viên triển khai trong môi trường AWS Learner Lab tại Region us-east-1 với thời hạn 4 tuần. Ứng dụng tích hợp Amazon S3 để lưu trữ web tĩnh, API Gateway tiếp nhận và xác thực yêu cầu, 4 hàm Lambda (Create, Get, Update, Delete) xử lý nghiệp vụ, và Amazon DynamoDB (kèm GSI major-index) lưu trữ cơ sở dữ liệu. Hệ thống được cấu hình bảo mật bằng API Key/Usage Plan và

giám sát qua Amazon CloudWatch. Báo cáo được tổng hợp từ nhật ký công việc từng tuần, mã nguồn triển khai và minh chứng hình ảnh thực tế trong repository.

2. Kiến trúc hệ thống

2.1. Sơ đồ và luồng hoạt động

Hệ thống được thiết kế theo kiến trúc serverless để cung cấp REST API CRUD cho tài nguyên /students. Giao diện web tĩnh được lưu trên Amazon S3. Các yêu cầu HTTP được gửi đến Amazon API Gateway, chuyển tiếp đến AWS Lambda để xử lý nghiệp vụ; Lambda tương tác với bảng Amazon DynamoDB. Amazon CloudWatch được sử dụng để lưu log và theo dõi hoạt động.



Hình 1. Sơ đồ kiến trúc hệ thống

Luồng xử lý:

- Người dùng truy cập giao diện tĩnh được lưu trữ trên Amazon S3.
- Frontend gửi yêu cầu GET, POST, PUT hoặc DELETE đến API Gateway cho tài nguyên /students.
- API Gateway kiểm tra API Key và áp dụng giới hạn lưu lượng theo Usage Plan, sau đó định tuyến yêu cầu đến Lambda tương ứng.
- Lambda xử lý nghiệp vụ bằng Python và đọc/ghi dữ liệu trong bảng Students trên DynamoDB.
- API Gateway và Lambda gửi log hoặc metrics đến CloudWatch. Nhà phát triển có thể dùng Postman để kiểm thử API độc lập với frontend.

2.2. Dịch vụ được lựa chọn và lý do

Dịch vụ	Vai trò	Lựa chọn
Amazon S3	Lưu trữ frontend tĩnh và cung cấp website.	Phù hợp với ứng dụng HTML/CSS/JavaScript, không cần duy trì web server.
Amazon API Gateway	Cung cấp REST API cho /students, nhận và định tuyến request.	Hỗ trợ các phương thức CRUD, API Key và Usage Plan để giới hạn mức sử dụng.
AWS Lambda	Chạy các hàm Python xử lý CRUD.	Thực thi theo yêu cầu, không cần quản lý máy chủ; tích hợp trực tiếp với API Gateway và DynamoDB.
Amazon DynamoDB	Lưu hồ sơ sinh viên trong bảng Students; dùng khóa chính student_id và GSI để truy vấn theo major.	Là cơ sở dữ liệu NoSQL phù hợp với cấu trúc dữ liệu của ứng dụng. Chế độ on-demand phù hợp yêu cầu đề tài và tính phí theo mức sử dụng.
IAM LabRole	Cấp quyền cho Lambda truy cập các tài nguyên cần thiết.	Learner Lab yêu cầu dùng role được cấp sẵn; không tạo IAM Role mới.
Amazon CloudWatch	Thu thập log và metrics của Lambda/API Gateway.	Hỗ trợ theo dõi request, lỗi và tình trạng hoạt động.
Postman	Gửi request để kiểm thử API.	Giúp kiểm tra từng endpoint mà không phụ thuộc vào giao diện web.

2.3. Ràng buộc của AWS Learner Lab

Thiết kế và triển khai cần tuân thủ các giới hạn trong đề tài:

- Region: Chỉ triển khai tại us-east-1 hoặc us-west-2. Dự án hiện ghi nhận sử dụng us-east-1; cần giữ các tài nguyên liên quan cùng Region.
- IAM: Dùng LabRole cho Lambda; không tạo IAM user hoặc IAM role mới.

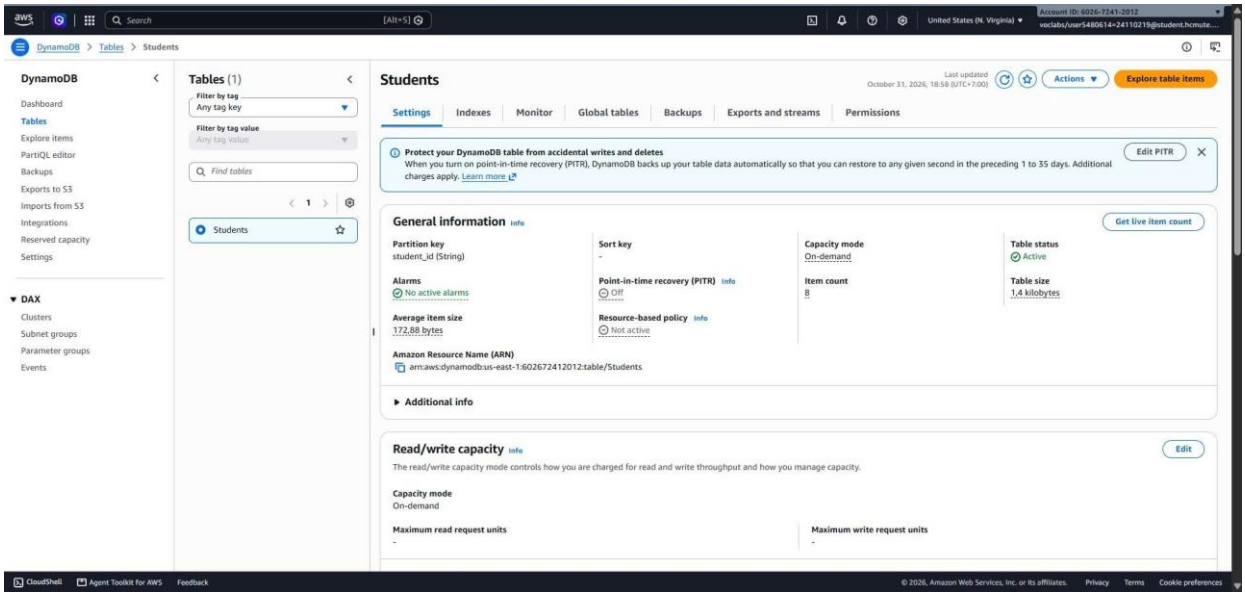
- Dịch vụ: Sử dụng API Gateway, Lambda, DynamoDB và S3 theo phạm vi dự án.
- DynamoDB: Dự án yêu cầu chế độ on-demand, giúp tính phí theo mức sử dụng thay vì cấu hình read/write capacity cố định.
- Giới hạn truy cập API: Bật API Key và Usage Plan như một biện pháp kiểm soát mức sử dụng. API Key đặt trong frontend công khai không phải cơ chế xác thực người dùng bí mật.
- Quản lý tài nguyên: Gắn tag Project và Owner cho các tài nguyên hỗ trợ tagging; dừng hoặc xóa tài nguyên không cần thiết sau mỗi buổi làm việc và tổng hợp chi phí cuối kỳ.

3. Triển khai hệ thống

Nhóm triển khai hệ thống theo từng tầng, từ cơ sở dữ liệu và backend đến REST API, giao diện web và giám sát. Các dịch vụ được cấu hình trên AWS Learner Lab, sử dụng Region us-east-1.

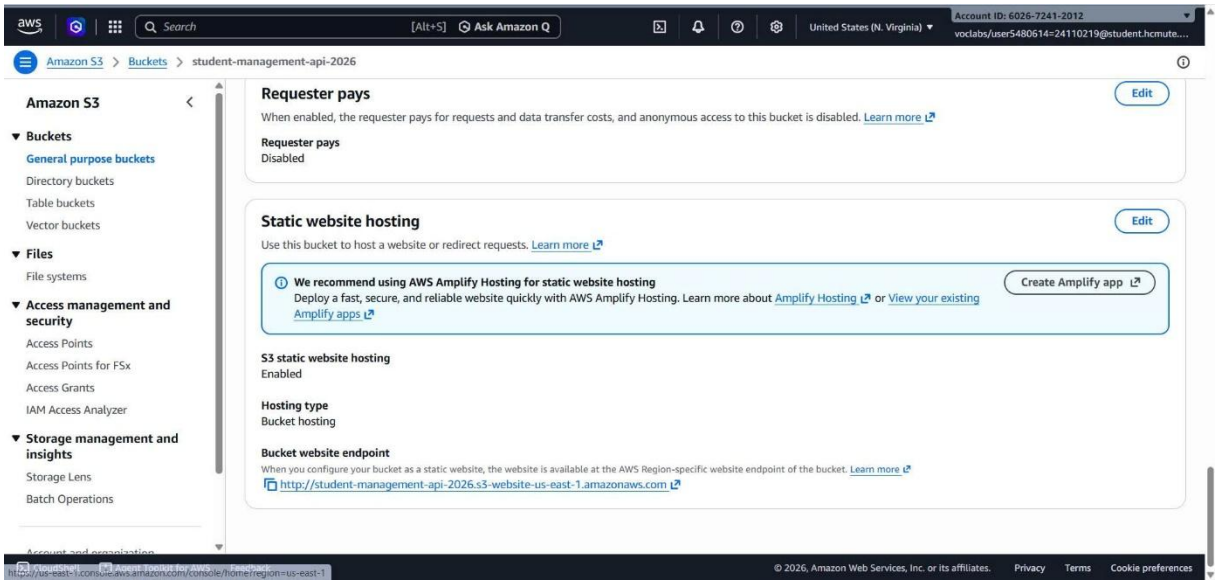
3.1. Khởi tạo cơ sở dữ liệu và lưu trữ giao diện

Nhóm tạo bảng DynamoDB Students với khóa chính student_id để định danh từng hồ sơ sinh viên. Để hỗ trợ tìm kiếm theo chuyên ngành, bảng có thêm Global Secondary Index tên major-index, sử dụng thuộc tính major làm khóa. Dữ liệu mẫu được chuẩn bị trong file cấu hình của dự án.



Hình 2. Cấu hình cơ sở dữ liệu trên DynamoDB

Amazon S3 được cấu hình để lưu trữ frontend tĩnh, với index.html làm trang mặc định. Nhóm thiết lập quyền đọc cần thiết để người dùng truy cập website qua S3 Static Website Hosting.



Hình 3. S3 Static Website Hosting

3.2. Xây dựng AWS Lambda

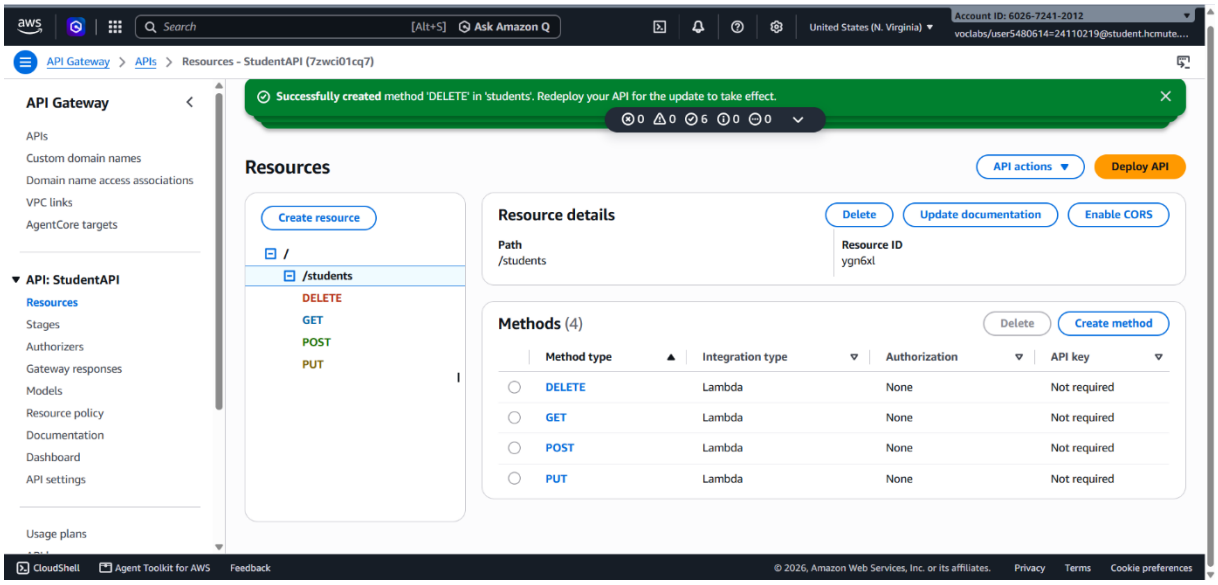
Nhóm xây dựng bốn hàm Lambda bằng Python để xử lý nghiệp vụ:

- CreateStudent: kiểm tra dữ liệu đầu vào, tạo hồ sơ mới và ghi thời điểm tạo/cập nhật vào DynamoDB.
- GetStudents: lấy toàn bộ danh sách, tìm theo student_id hoặc truy vấn theo major thông qua GSI major-index.
- UpdateStudent: kiểm tra dữ liệu và cập nhật các trường được gửi lên.
- DeleteStudent: xóa hồ sơ theo student_id.

Các hàm trả về HTTP status code và nội dung JSON phù hợp với kết quả xử lý. Nhóm cũng bổ sung kiểm tra các trường bắt buộc và định dạng dữ liệu như email, ngày sinh, GPA và trạng thái sinh viên. Với thao tác cập nhật hoặc xóa, điều kiện tồn tại của bản ghi giúp API trả về 404 khi không tìm thấy sinh viên thay vì báo thành công. Các hàm sử dụng IAM LabRole để truy cập DynamoDB, không tạo IAM Role mới.

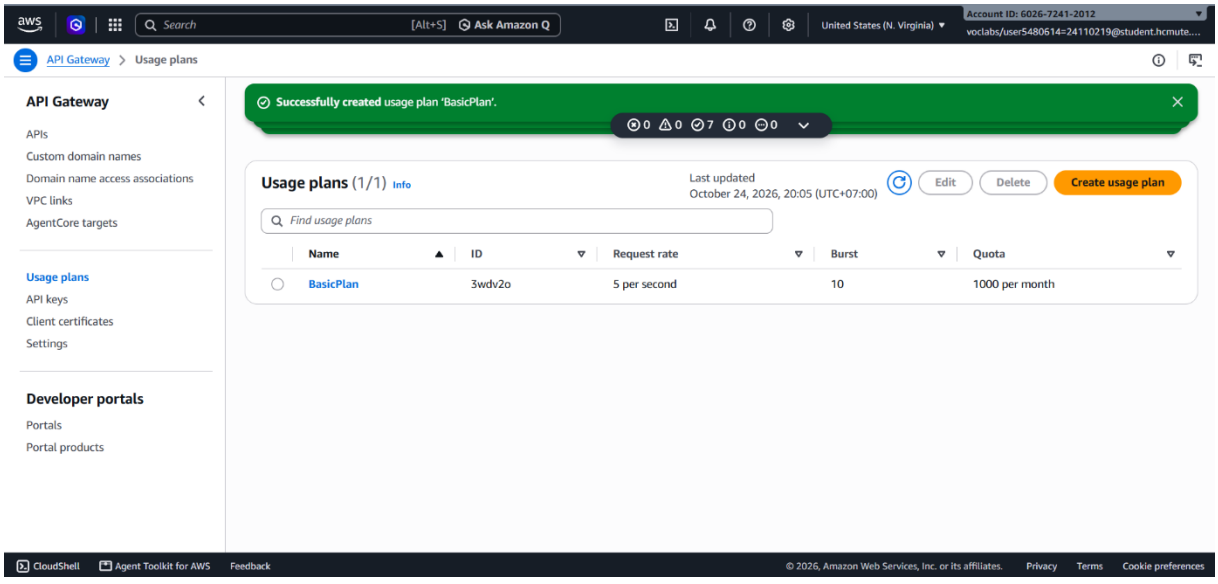
3.3. Tạo REST API bằng Amazon API Gateway

Nhóm tạo REST API StudentAPI với resource /students. Các phương thức GET, POST, PUT và DELETE được cấu hình để chuyển request đến các hàm Lambda tương ứng thông qua Lambda Proxy Integration. Nhóm bật CORS để frontend trên S3 có thể gọi API khác nguồn, sau đó triển khai API lên stage dev.



Hình 4. Cấu hình Student API trên API Gateway

Usage Plan BasicPlan được cấu hình Rate 5 request/giây, Burst 10 request và Quota 1.000 request/tháng. Frontend gửi API Key qua header x-api-key.

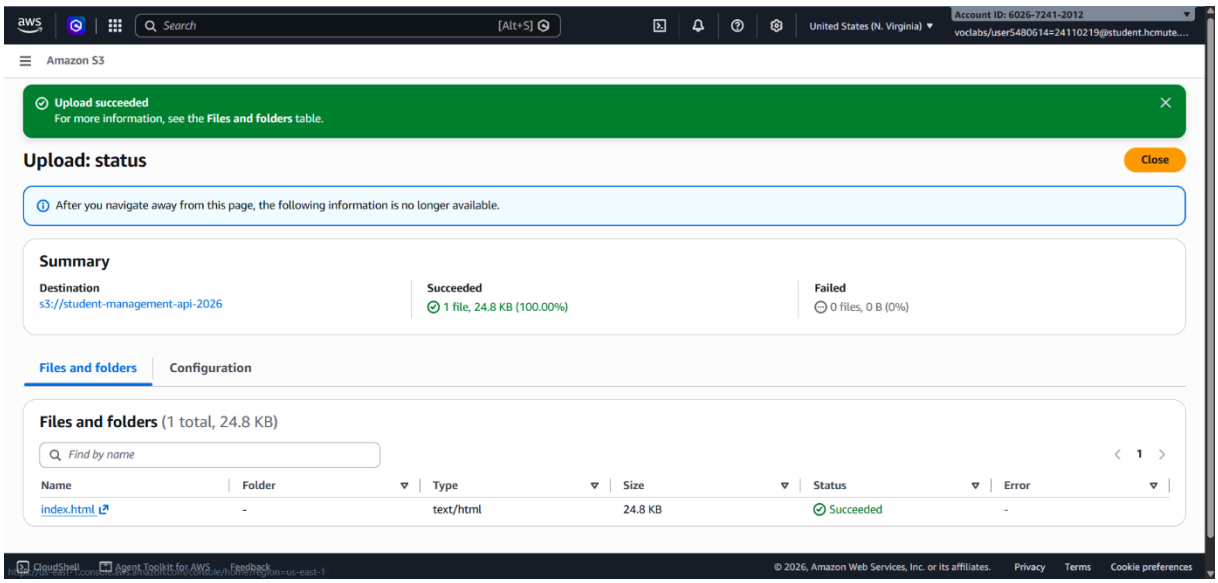


Hình 5. Tạo Usage Plan BasicPlan

3.4. Tích hợp frontend với API

Frontend được xây dựng trong file index.html bằng HTML, CSS và JavaScript. Người dùng có thể nhập API Gateway Invoke URL và API Key trên giao diện. Frontend gửi request bằng Fetch API để thực hiện các thao tác CRUD, hiển thị danh sách sinh viên và lọc theo mã sinh viên hoặc chuyên ngành.

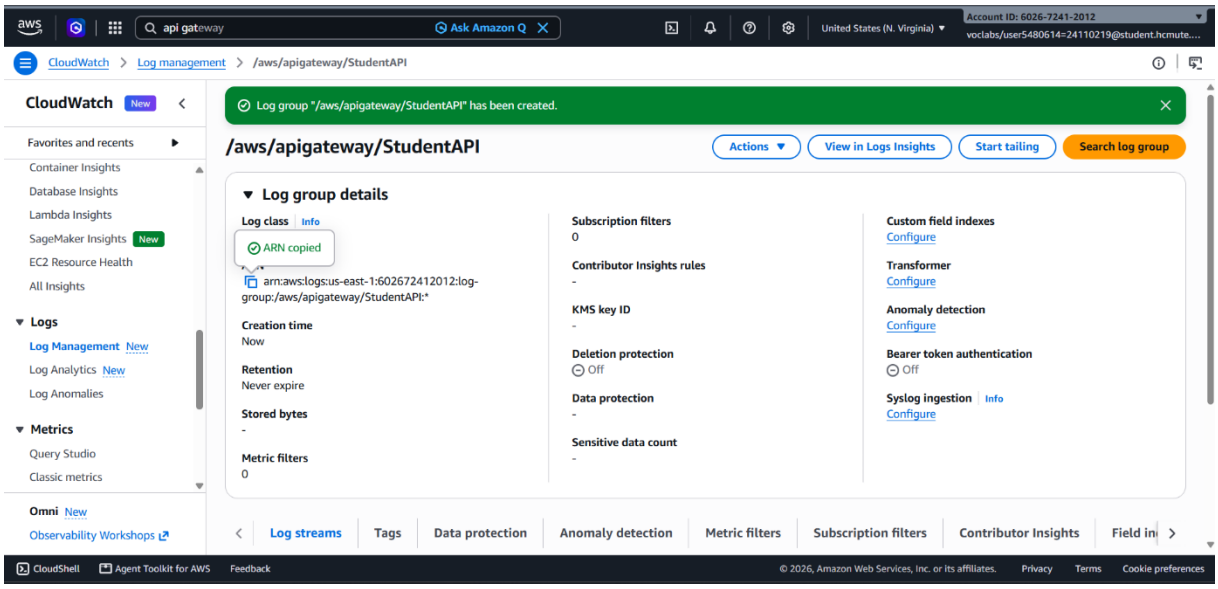
Nhóm tải frontend lên S3 Static Website Hosting và kiểm thử trực tiếp trên website, thay vì chỉ chạy tệp cục bộ. Khi thao tác thành công, giao diện tải lại danh sách để phản ánh dữ liệu mới.



Hình 6. Tải file index.html lên S3

3.5. Thiết lập giám sát và kiểm thử

Nhóm tạo CloudWatch Log Group cho API Gateway và bật Access Logging tại stage dev. Log được định dạng JSON để ghi nhận các thông tin request như thời gian, IP nguồn, HTTP method và status code. Nhóm cũng theo dõi các metrics như số lượng request và độ trễ của API.



Hình 7. Tạo CloudWatch Log Group

Kiểm thử được thực hiện ở nhiều mức: kiểm thử từng hàm Lambda trên AWS Console, kiểm thử endpoint bằng Postman, và kiểm thử end-to-end qua giao diện S3. Các kịch bản bao gồm CRUD thành công, tìm kiếm theo mã/ngành, trùng mã sinh viên, dữ liệu sai định dạng, GPA ngoài phạm vi và cập nhật/xóa mã không tồn tại.

4. Bảo mật

Nhóm triển khai các biện pháp bảo mật theo phạm vi đồ án và các dịch vụ được phép sử dụng trong AWS Learner Lab. Các biện pháp tập trung vào quản lý quyền truy cập bằng IAM LabRole, kiểm soát request đến API Gateway bằng API Key và Usage Plan, đồng thời giới hạn quyền truy cập công khai vào S3 ở mức chỉ đọc website.

4.1. Quản lý quyền truy cập bằng IAM

Các hàm AWS Lambda sử dụng IAM Role LabRole được cấp sẵn trong AWS Learner Lab để truy cập bảng DynamoDB và thực hiện các thao tác cần thiết. API Gateway cũng sử dụng LabRole để ghi access log lên CloudWatch. Nhóm không tạo IAM user hoặc IAM role mới, tuân thủ yêu cầu của môi trường học tập.

Việc sử dụng IAM Role giúp dịch vụ AWS xác thực và cấp quyền cho Lambda mà không cần lưu thông tin đăng nhập AWS trong mã nguồn. Nhóm sử dụng role được cung cấp sẵn thay vì tự tạo role riêng trong phạm vi Learner Lab.

4.2. Bảo mật mạng và kiểm soát truy cập

API Gateway là điểm tiếp nhận request từ bên ngoài Internet. Các thao tác CRUD trên tài nguyên /students yêu cầu API Key trong header x-api-key. API Key được liên kết với Usage Plan BasicPlan, giới hạn lưu lượng ở mức 5 request/giây, Burst 10 request và Quota 1.000 request/tháng. Cấu hình này giúp quản lý mức sử dụng API và hạn chế request vượt quá ngưỡng đã đặt.

Giao diện web được lưu trên Amazon S3 Static Website Hosting. Bucket Policy cấp quyền s3:GetObject công khai để người dùng tải các tệp giao diện; policy không cấp quyền ghi hoặc xóa đối tượng. CORS được cấu hình trên API Gateway để cho phép trình duyệt gửi request từ website S3 đến API. CORS chỉ cho phép hoặc giới hạn request cross-origin từ trình duyệt, không thay thế cơ chế xác thực API.

4.3. Bảo vệ API Key và thông tin nhạy cảm

API Key được nhập thông qua giao diện website và gửi đến API Gateway bằng HTTP Header `x-api-key`. Frontend lưu trữ API Key trong `sessionStorage` của trình duyệt để sử dụng trong phiên làm việc hiện tại. Giá trị API Key không được khai báo cố định trong mã nguồn frontend.

Do website được công khai và mã JavaScript phía frontend được thực thi trực tiếp trên trình duyệt người dùng, API Key được sử dụng tại đây có thể bị người dùng truy cập. Vì vậy, trong phạm vi đồ án, API Key chỉ đóng vai trò kiểm soát và giới hạn mức sử dụng API, không được xem là thông tin bí mật hoặc cơ chế xác thực người dùng có độ bảo mật cao. Nhóm không đưa giá trị API Key thực tế vào mã nguồn công khai.

4.4. Tổng kết

Các biện pháp bảo mật được triển khai phù hợp với mục tiêu học tập và giới hạn của AWS Learner Lab: sử dụng LabRole, yêu cầu API Key, giới hạn request bằng Usage Plan, giới hạn quyền công khai của S3 ở mức đọc website và ghi log API lên CloudWatch. Những hạn chế của mô hình gồm website S3 sử dụng HTTP và API Key trên frontend có thể bị người truy cập xem; các điểm này được xem xét khi đánh giá phạm vi bảo mật của hệ thống.

5. Kiểm thử và giám sát hệ thống

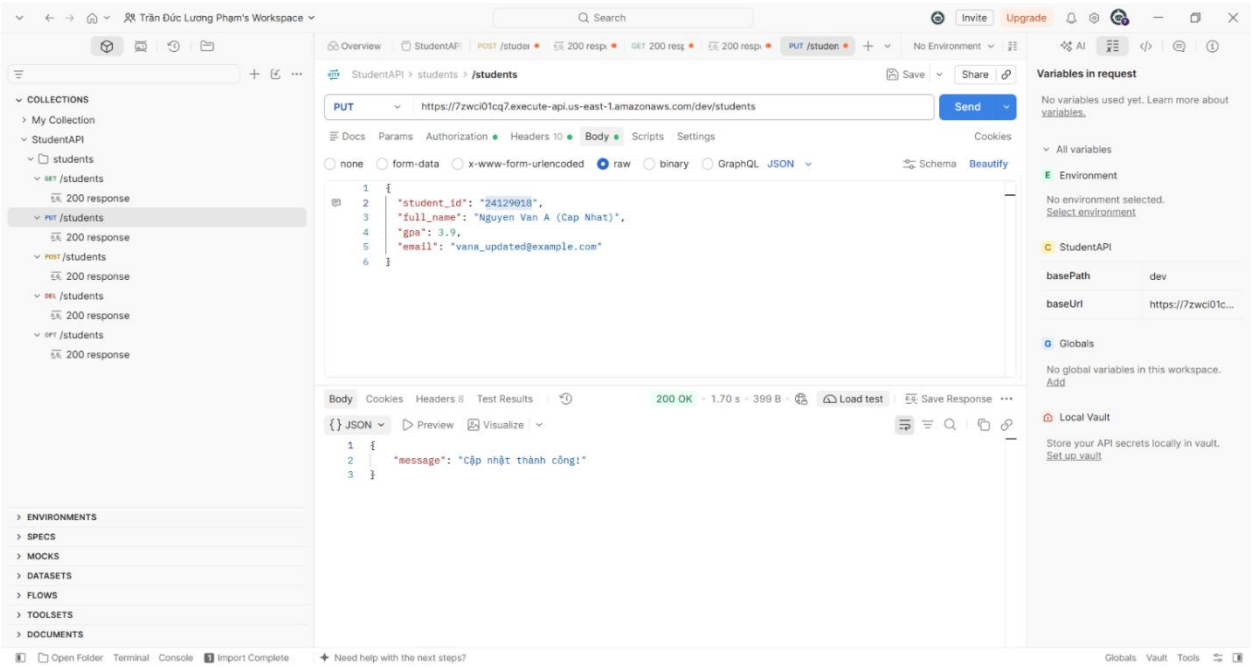
5.1. Kiểm thử chức năng API

Sau khi hoàn thiện các hàm AWS Lambda và triển khai trên Amazon API Gateway, nhóm tiến hành kiểm thử nhằm đánh giá tính chính xác của các chức năng quản lý sinh viên, khả năng xử lý dữ liệu đầu vào và phản hồi của hệ thống khi phát sinh lỗi nghiệp vụ.

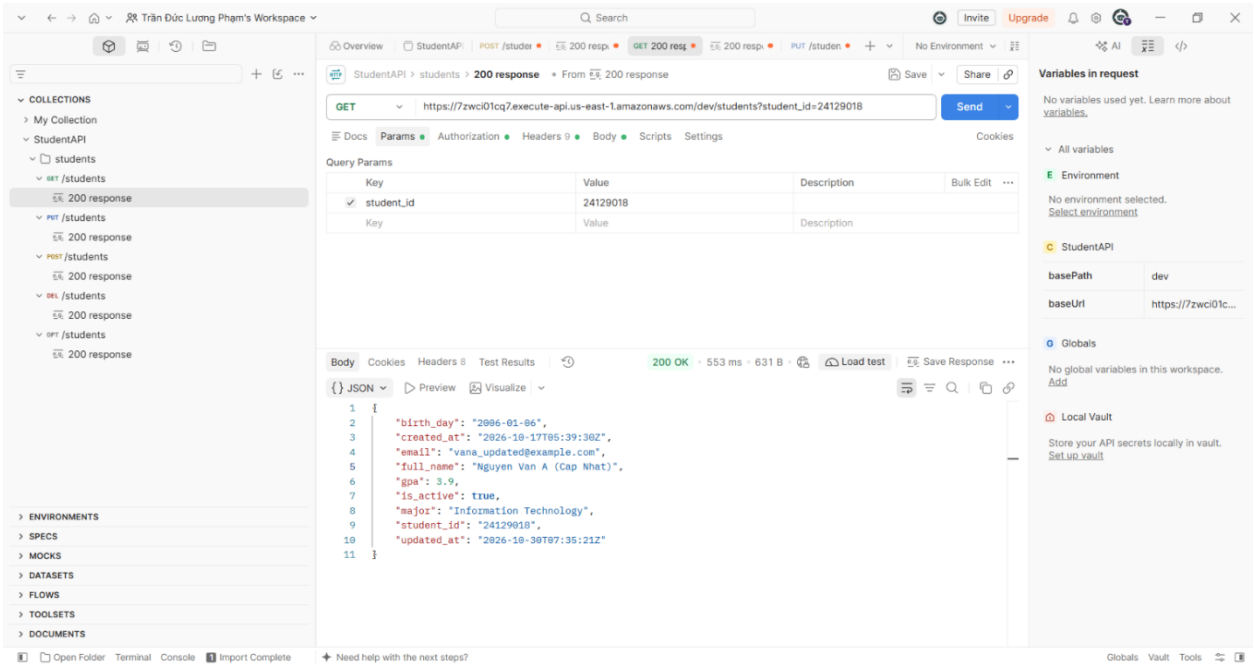
Nhóm sử dụng Postman để thực hiện kiểm thử độc lập các endpoint. Từ tài liệu đặc tả OpenAPI 3 được xuất từ API Gateway, nhóm xây dựng Postman Collection, cấu hình tự động gửi API Key thông qua HTTP Header x-api-key và thiết lập các dữ liệu đầu vào phục vụ quá trình kiểm thử.

Trong quá trình kiểm thử, nhóm tập trung vào hai nhóm trường hợp chính:

Kiểm thử chức năng thành công: Xác nhận các thao tác tạo, truy vấn, cập nhật và xóa sinh viên hoạt động chính xác, dữ liệu được ghi nhận hoặc thay đổi tương ứng trên DynamoDB.

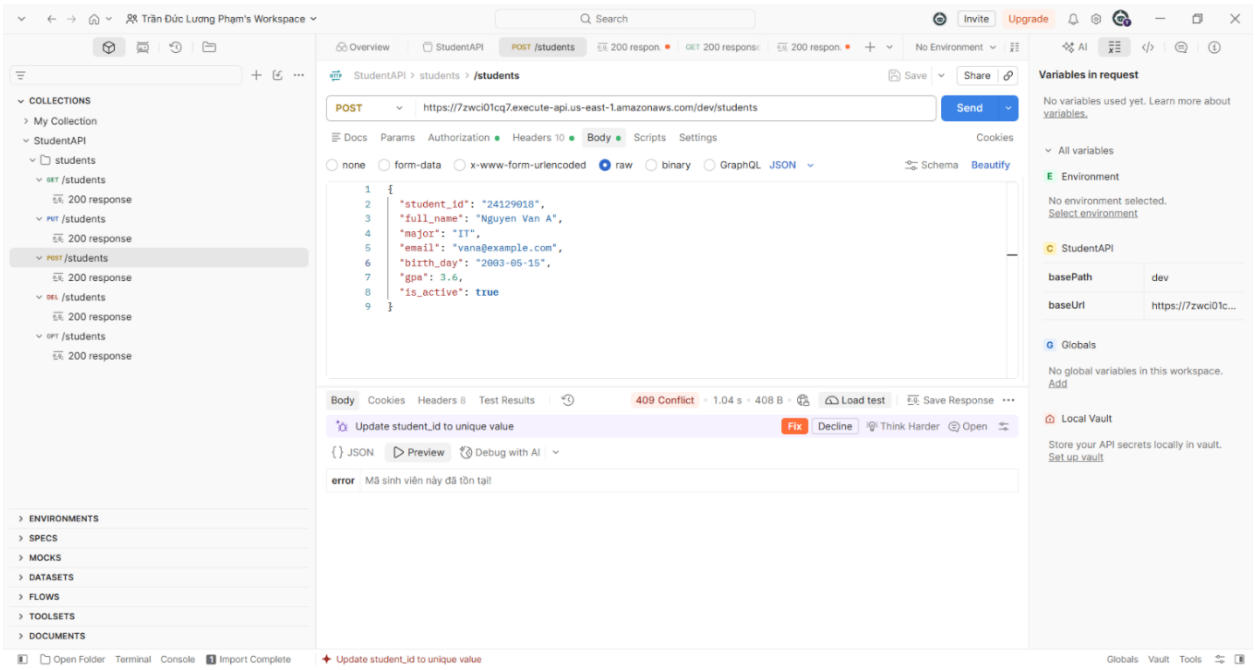


Hình 8. Postman cập nhật sinh viên thành công



Hình 9. Postman kiểm tra lại thông tin sau cập nhật

Kiểm thử ngoại lệ nghiệp vụ: Kiểm tra khả năng phát hiện dữ liệu không hợp lệ, trường thông tin bắt buộc bị thiếu, mã sinh viên không tồn tại hoặc dữ liệu có định dạng không đúng. Hệ thống cần trả về mã phù hợp cùng thông báo lỗi để hỗ trợ người dùng xác định nguyên nhân.

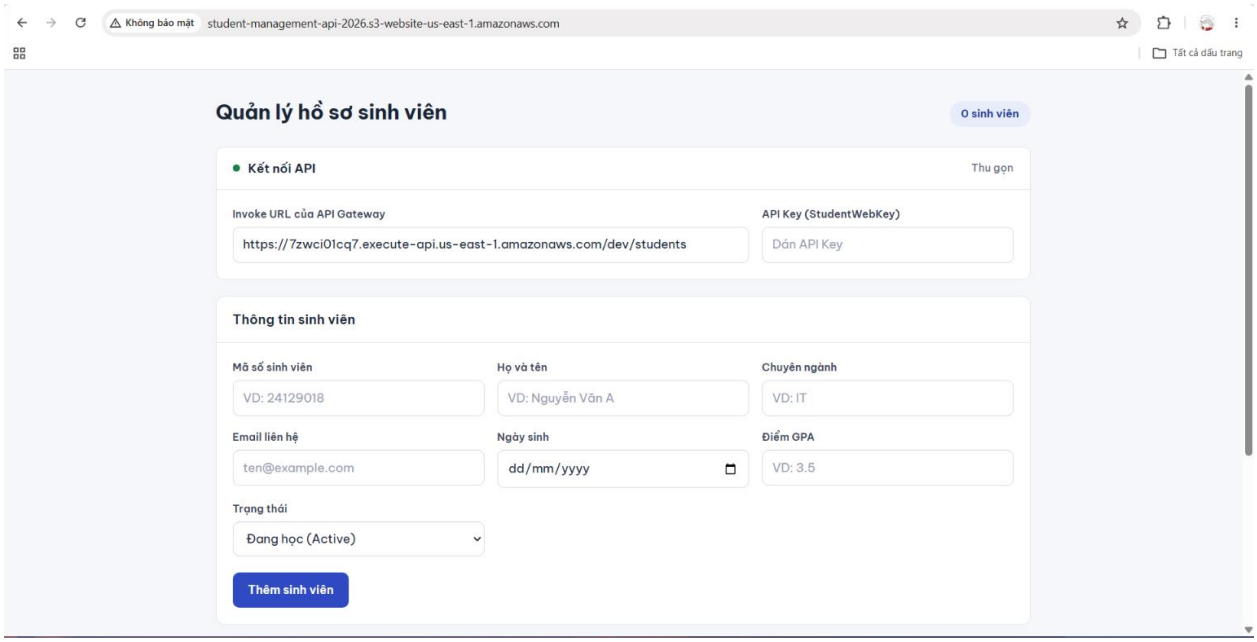


Hình 10. Postman thêm sinh viên với mã đã tồn tại

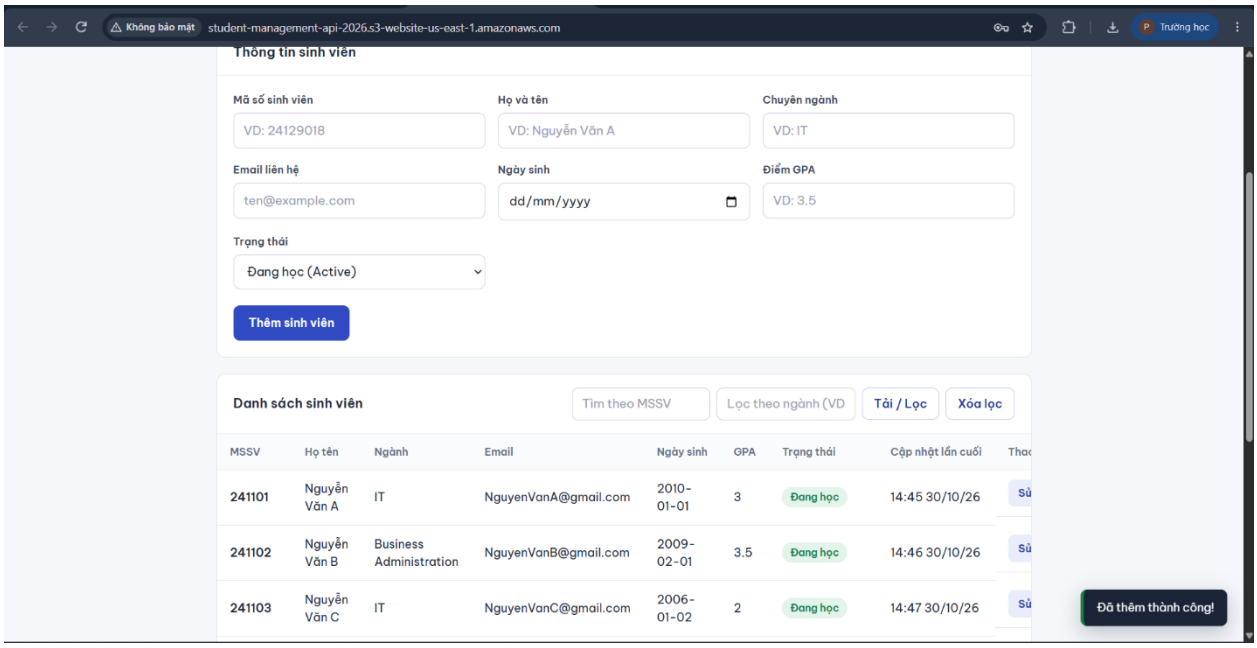
5.2. Kiểm thử tích hợp End-to-End

Tiến hành kiểm thử thao tác trên giao diện website tĩnh nhằm đánh giá khả năng phối hợp giữa các thành phần trong toàn bộ kiến trúc hệ thống.

Các thao tác bao gồm thêm mới, cập nhật, xóa và lọc danh sách sinh viên. Sau mỗi thao tác, hệ thống gửi yêu cầu đến API Gateway, xử lý thông qua Lambda và cập nhật dữ liệu trên DynamoDB. Kết quả phản hồi được hiển thị trên giao diện thông qua danh sách sinh viên và các thông báo tương ứng.

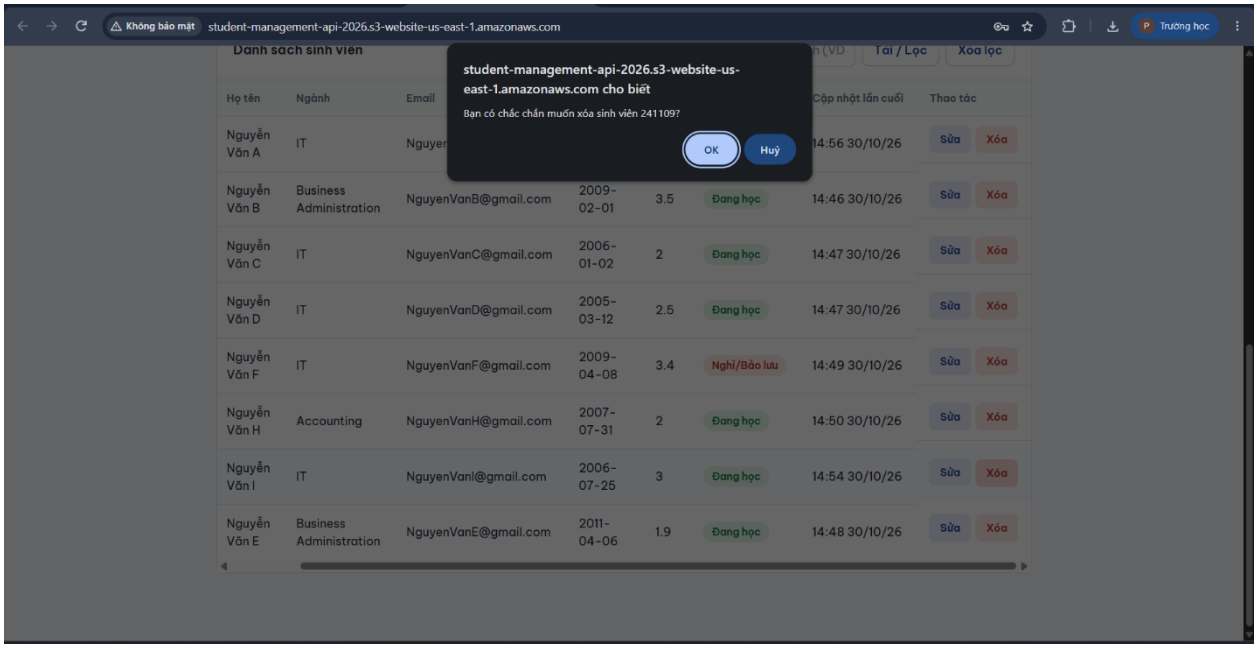


Hình 11. Giao diện trang web

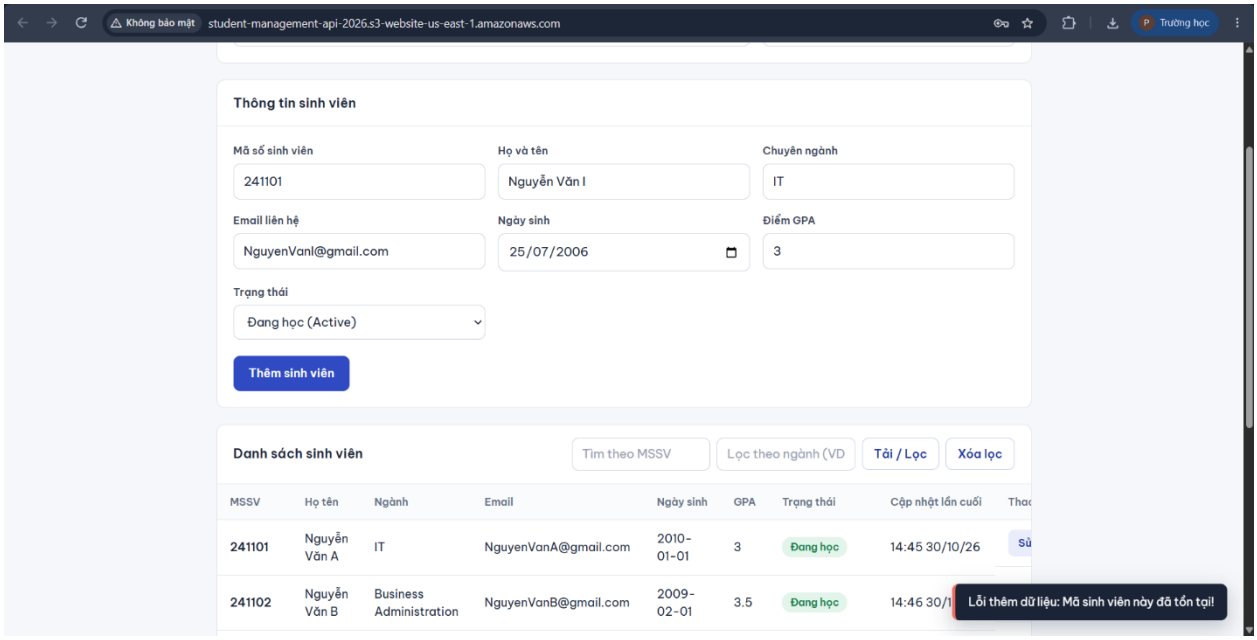


Hình 12. Kiểm thử thêm sinh viên thành công

Bên cạnh đó, nhóm bổ sung các thông báo xác nhận trước khi thực hiện thao tác xóa và thông báo lỗi khi dữ liệu đầu vào không đáp ứng các điều kiện nghiệp vụ như trùng mã sinh viên, sai định dạng email hoặc GPA không hợp lệ.



Hình 13. Thông báo xác nhận khi xóa

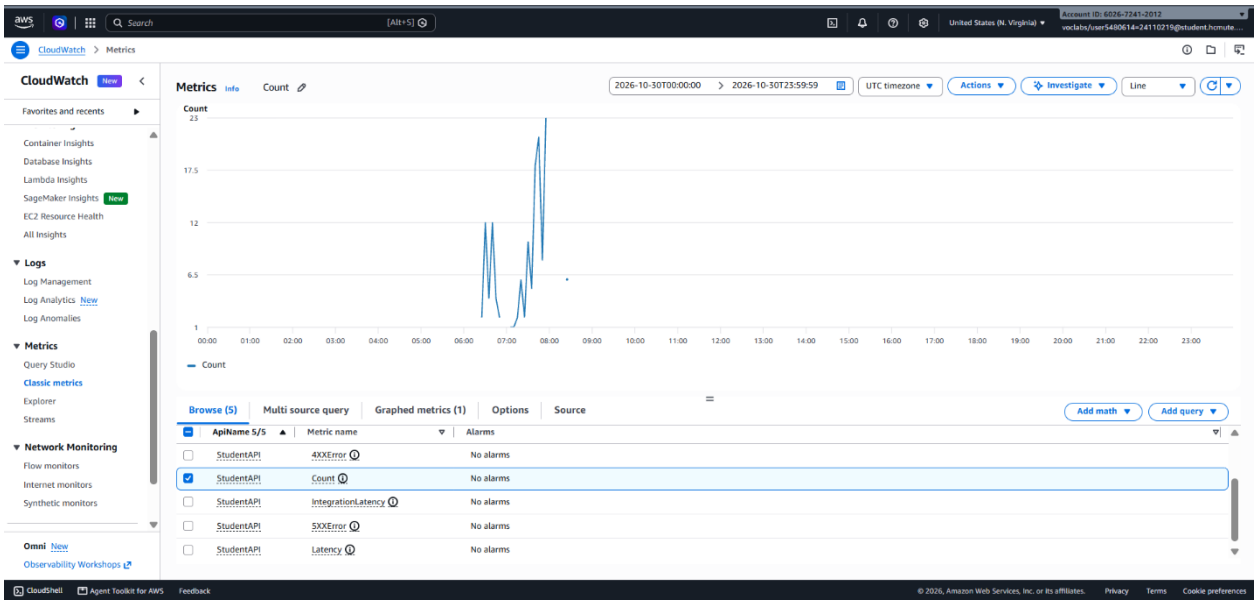


Hình 14. Lỗi thêm sinh viên mã đã tồn tại

5.3. Giám sát hệ thống

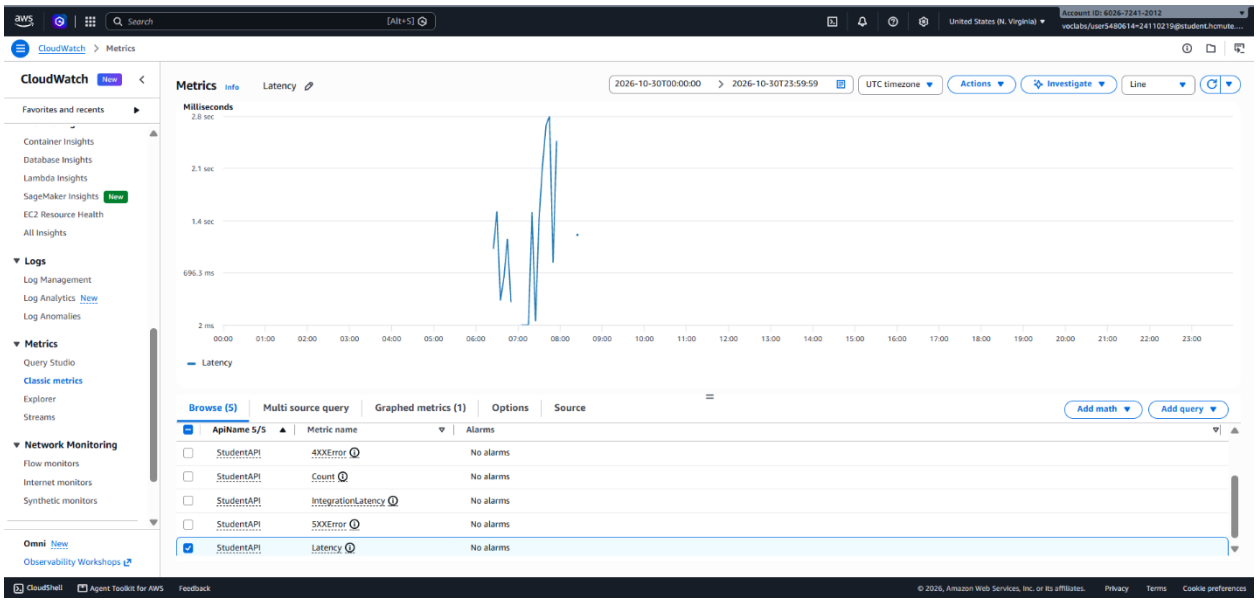
Nhóm sử dụng dịch vụ Amazon CloudWatch để theo dõi hoạt động và đánh giá hiệu suất của REST API trong quá trình kiểm thử thông qua CloudWatch Metrics. Đối với API StudentAPI, nhóm theo dõi các chỉ số:

Count: Thống kê số lượng yêu cầu API được xử lý trong từng khoảng thời gian, giúp đánh giá mức độ sử dụng hệ thống.



Hình 15. Biểu đồ Count

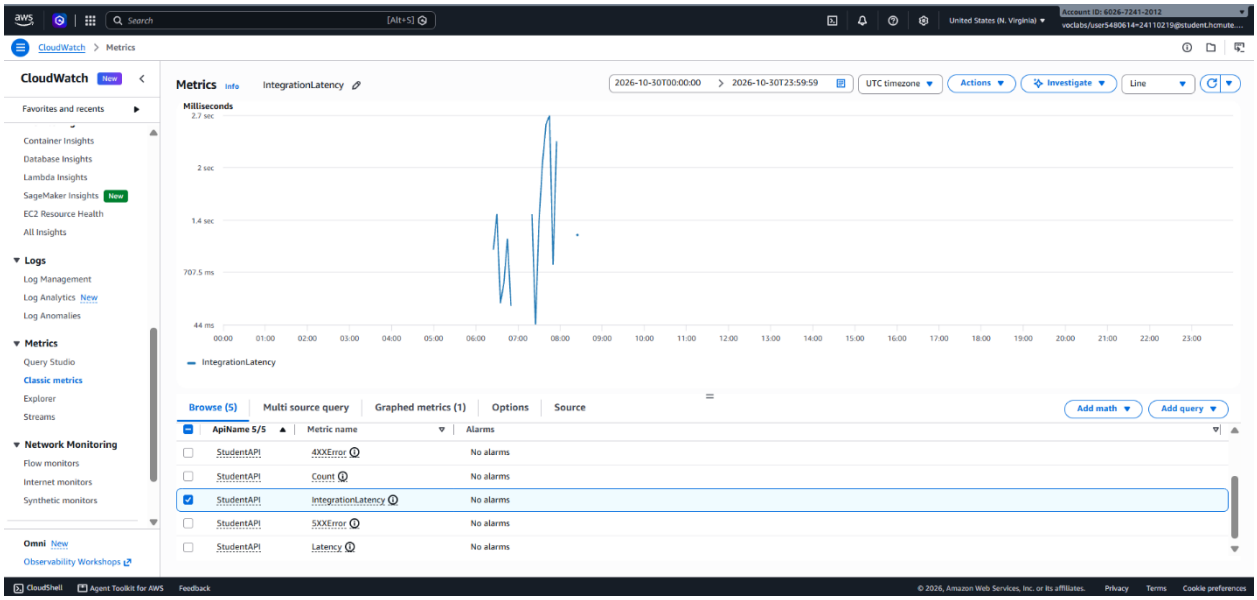
Latency: Thể hiện tổng thời gian xử lý một yêu cầu API, tính từ khi API Gateway nhận yêu cầu đến khi trả kết quả về phía người dùng.



Hình 16. Biểu đồ Latency

Biểu đồ Latency ghi nhận thời gian phản hồi của API có sự dao động trong khoảng thời gian phát sinh yêu cầu. Giá trị cao nhất trên biểu đồ xấp xỉ 2,8 giây. Sự thay đổi này có thể liên quan đến quá trình xử lý yêu cầu tại API Gateway và các dịch vụ backend.

IntegrationLatency: Thể hiện thời gian API Gateway chờ dịch vụ backend xử lý yêu cầu và phản hồi, qua đó hỗ trợ đánh giá thời gian xử lý tại tầng tích hợp.



Hình 17. Biểu đồ IntegrationLatency

Chỉ số IntegrationLatency có giá trị cao nhất trên biểu đồ khoảng 2,7 giây. Việc theo dõi chỉ số này giúp nhóm có thêm cơ sở đánh giá thời gian xử lý của các hàm AWS Lambda khi tiếp nhận yêu cầu từ API Gateway.

Ngoài ra, CloudWatch còn giúp giám sát 4XXError (số lượng lỗi thuộc nhóm mã HTTP 4XX liên quan đến dữ liệu đầu vào không hợp lệ) và 5XXError (số lượng lỗi thuộc nhóm mã HTTP 5XX phát sinh trong quá trình xử lý phía máy chủ hoặc backend).

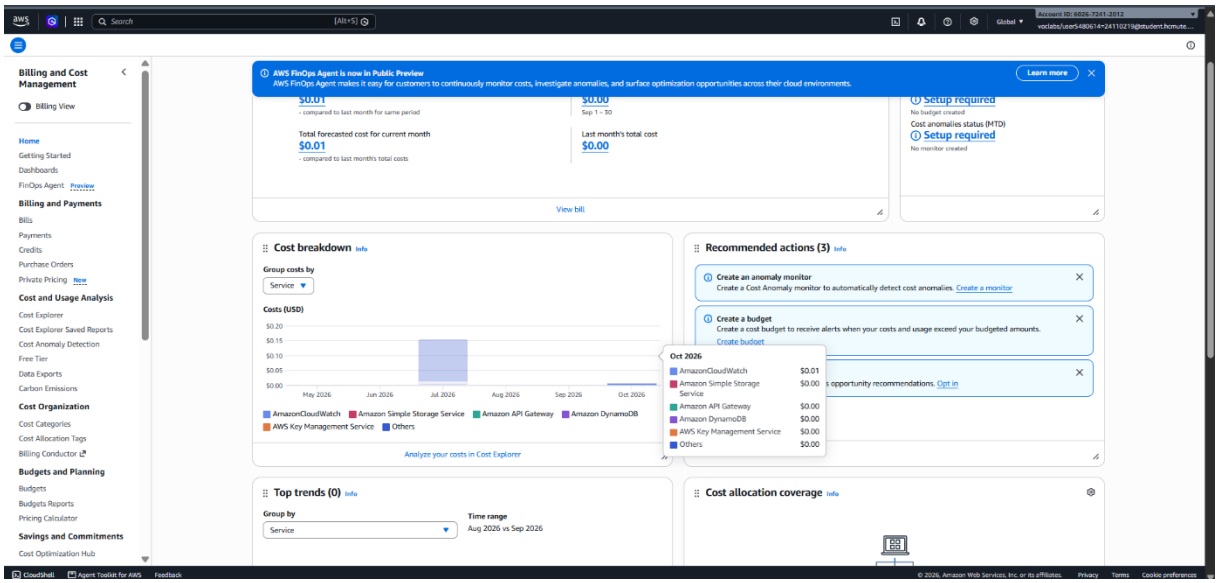
6. Chi phí và tối ưu hóa

6.1. Chi phí ghi nhận

Nhóm sử dụng Billing and Cost Management và Cost Explorer để theo dõi chi phí AWS. Theo ảnh chụp Cost Explorer trong khoảng thời gian 01–31/10/2026, tổng chi phí hiển thị là 0,01 USD. Ảnh tổng quan Billing cũng hiển thị chi phí dự báo của tháng

là 0,01 USD. Trong bảng phân bổ theo dịch vụ, CloudWatch hiển thị khoảng 0,01 USD; S3, API Gateway, DynamoDB và AWS KMS hiển thị 0,00 USD tại thời điểm chụp.

Các số liệu trên phản ánh dữ liệu hiển thị trong AWS Console tại thời điểm chụp, không nhất thiết là hóa đơn cuối cùng của toàn bộ thời gian thực hiện đồ án. Nhóm cần đối chiếu Billing sau khi kết thúc triển khai để tổng hợp số liệu cuối kỳ.



Hình 18. Billing and Cost Management

6.2. Các biện pháp tối ưu chi phí

Nhóm lựa chọn kiến trúc Serverless với các dịch vụ Amazon S3, Amazon API Gateway, AWS Lambda và Amazon DynamoDB nhằm tối ưu chi phí vận hành hệ thống. Với kiến trúc này, nhóm không cần duy trì máy chủ ứng dụng hoạt động liên tục. AWS Lambda chỉ thực thi khi có request, giúp hạn chế việc sử dụng tài nguyên khi hệ thống không phát sinh yêu cầu.

Đối với API Gateway, nhóm sử dụng Usage Plan để giới hạn mức sử dụng API, cụ thể là 5 request/giây, Burst 10 request và tối đa 1.000 request/tháng. Việc thiết lập các giới hạn này giúp kiểm soát lưu lượng truy cập và hạn chế số lượng request vượt quá nhu cầu sử dụng của hệ thống.

Nhóm sử dụng Amazon CloudWatch để theo dõi log và các chỉ số hoạt động, hỗ trợ giám sát quá trình xử lý request, hoạt động của Lambda và phát hiện các vấn đề phát sinh. Đối với DynamoDB, nhóm lựa chọn chế độ tính phí **On-Demand** nhằm đáp ứng nhu cầu đọc và ghi dữ liệu linh hoạt mà không cần thiết lập trước dung lượng xử lý.

Chế độ này phù hợp với hệ thống có lưu lượng truy cập không ổn định và quy mô kiểm thử của đề tài, trong đó chi phí được tính dựa trên mức độ sử dụng thực tế.

Bên cạnh đó, nhóm thiết kế Global Secondary Index (GSI) có tên `major-index` nhằm hỗ trợ truy vấn dữ liệu sinh viên theo chuyên ngành. Tuy nhiên, chức năng lấy toàn bộ danh sách sinh viên hiện sử dụng thao tác `Scan`, do đó nhóm cần chú ý đến số lượng dữ liệu và tần suất truy vấn để hạn chế lượng dữ liệu đọc không cần thiết khi hệ thống mở rộng.

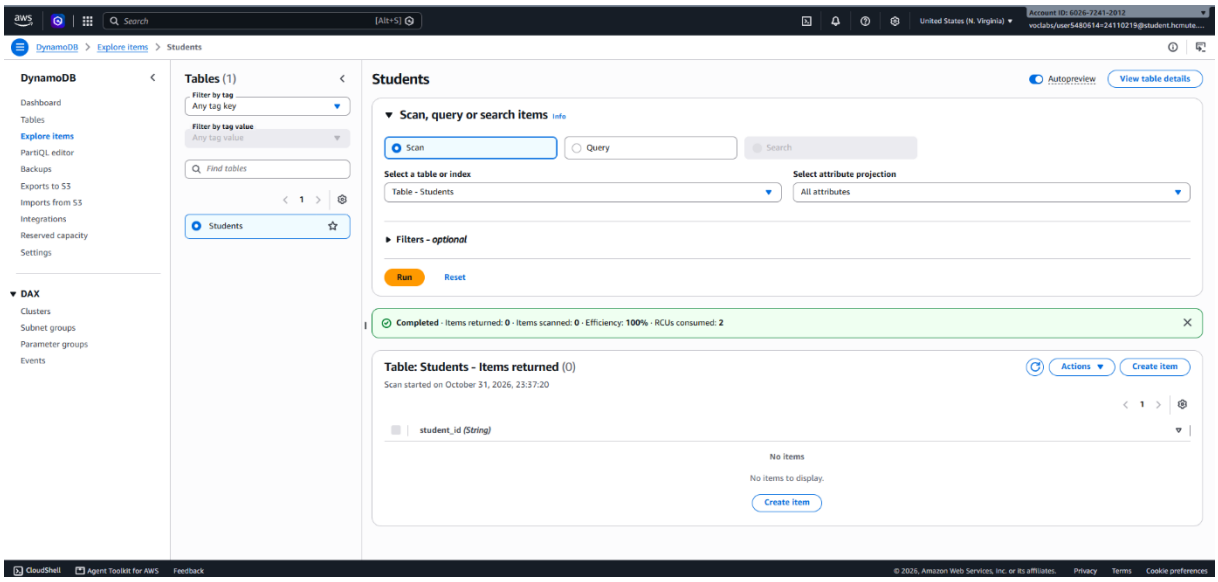
Cuối cùng, nhóm thực hiện kiểm tra và dọn dẹp các tài nguyên AWS không còn sử dụng sau mỗi buổi thực hành, đồng thời duy trì những tài nguyên cần thiết cho quá trình phát triển và kiểm thử. Qua đó, nhóm hướng đến việc sử dụng tài nguyên hợp lý, hạn chế chi phí phát sinh và đảm bảo hiệu quả vận hành hệ thống.

6.3. Quản lý và dọn dẹp tài nguyên

Trong quá trình thực hiện đồ án, nhóm thường xuyên kiểm tra và quản lý các tài nguyên AWS sau mỗi buổi làm việc nhằm hạn chế việc sử dụng tài nguyên không cần thiết. Sau khi hoàn thành các buổi thực hành, nhóm đóng phiên Learner Lab và tiến hành rà soát các tài nguyên đã triển khai.

Đối với hệ thống chính, nhóm duy trì các tài nguyên cần thiết như API Gateway, AWS Lambda, bảng và GSI DynamoDB, Amazon S3 cùng các cấu hình liên quan để phục vụ quá trình phát triển và kiểm thử. Các dữ liệu thử nghiệm không còn cần thiết được xóa khỏi bảng DynamoDB nhằm tránh lưu trữ dữ liệu dư thừa. Đồng thời, nhóm kiểm tra và loại bỏ các tài nguyên thử nghiệm không còn sử dụng.

Sau khi hoàn thành đồ án, nhóm tiến hành rà soát tổng thể các tài nguyên AWS, kiểm tra trạng thái hoạt động và dọn dẹp những tài nguyên không còn cần thiết. Các tài nguyên và dữ liệu cần thiết cho việc lưu trữ mã nguồn, báo cáo và minh chứng được giữ lại. Việc kiểm tra và dọn dẹp được thực hiện nhằm hạn chế chi phí phát sinh, tránh lãng phí tài nguyên và đảm bảo việc sử dụng AWS Learner Lab đúng mục đích.



Hình 19. Xóa tài nguyên

7. Những vấn đề gặp phải và bài học kinh nghiệm

7.1. Các vấn đề gặp phải

Xử lý hành vi mặc định của Amazon DynamoDB khi cập nhật và xóa dữ liệu.

Trong quá trình xây dựng các hàm UpdateStudent và DeleteStudent, nhóm nhận thấy thao tác cập nhật bằng UpdateItem có thể tự động tạo một bản ghi mới nếu khóa chính chưa tồn tại. Trong khi đó, thao tác xóa bằng DeleteItem có thể không phát sinh lỗi ngay cả khi bản ghi cần xóa không tồn tại. Điều này dẫn đến nguy cơ API trả về kết quả thành công trong khi mã sinh viên được yêu cầu cập nhật hoặc xóa thực tế không tồn tại trong cơ sở dữ liệu.

Để khắc phục, nhóm bổ sung điều kiện vào các thao tác cập nhật và xóa nhằm đảm bảo chỉ xử lý những bản ghi đã tồn tại. Đồng thời, nhóm sử dụng ConditionalCheckFailedException để nhận diện trường hợp điều kiện không thỏa mãn và trả về mã HTTP 404. Sau khi điều chỉnh, hệ thống có thể phân biệt rõ giữa thao tác thành công và trường hợp không tìm thấy hồ sơ sinh viên, giúp nâng cao tính chính xác của phản hồi API.

Quản lý quyền truy cập và cấu hình trong môi trường AWS Learner Lab.

Trong quá trình triển khai, nhóm phải tuân thủ các giới hạn tài nguyên và quyền truy cập của AWS Academy Learner Lab. Đặc biệt, môi trường không cho phép tự tạo

IAM Role mới mà yêu cầu sử dụng các quyền được cấp sẵn. Nhóm sử dụng LabRole để cấp quyền truy cập cần thiết cho các hàm Lambda và cấu hình phục vụ việc ghi nhật ký. Đồng thời, nhóm thống nhất sử dụng Region us-east-1 cho các tài nguyên nhằm đảm bảo sự đồng bộ trong quá trình triển khai. Qua đó, nhóm rút ra kinh nghiệm cần tìm hiểu kỹ các giới hạn của môi trường thực hành trước khi thiết kế và triển khai kiến trúc, tránh phát sinh lỗi do sử dụng dịch vụ hoặc quyền không được hỗ trợ.

7.2. Bài học kinh nghiệm

Qua các tuần thực hiện dự án, nhóm rút ra một số bài học kinh nghiệm sau:

- Về thiết kế cơ sở dữ liệu: Cần hiểu rõ hành vi của các thao tác trên DynamoDB, đặc biệt là sự khác biệt giữa cập nhật dữ liệu và tạo mới bản ghi. Việc sử dụng điều kiện khi cập nhật hoặc xóa giúp đảm bảo tính chính xác của các thao tác nghiệp vụ.
- Về phát triển Backend: Cần xây dựng cơ chế kiểm tra dữ liệu đầu vào và xử lý ngoại lệ ngay trong các hàm Lambda. Các mã trạng thái HTTP phải phản ánh đúng kết quả xử lý để Frontend có thể tiếp nhận và hiển thị thông báo phù hợp.
- Về tích hợp hệ thống: Cần kiểm thử từng thành phần trước khi tiến hành kiểm thử toàn bộ luồng hoạt động. Việc kết hợp Postman và kiểm thử E2E giúp phát hiện vấn đề ở cả cấp độ API và giao diện người dùng.
- Về bảo mật và vận hành: Cần hiểu rõ cơ chế phân quyền IAM, cách sử dụng API Key, Usage Plan và vai trò của CloudWatch trong việc theo dõi hoạt động hệ thống.
- Về quản lý tài nguyên Cloud: Cần chú trọng việc lựa chọn cấu hình phù hợp, theo dõi chi phí và tuân thủ các giới hạn của môi trường AWS Learner Lab trong suốt quá trình triển khai.

8. Đóng góp của các thành viên và khai báo sử dụng AI

8.1. Đóng góp của các thành viên

Đồ án được thực hiện theo từng tuần, mỗi thành viên trong nhóm phải tham gia thực hiện tối thiểu 8 giờ/tuần. Cụ thể công việc thực hiện như sau:

Tuần 1: Tổng thời gian: Phan Nguyễn Quốc Hoàng thực hiện 8 giờ, Phạm Trần Đức Lương thực hiện 8 giờ.

Thành viên	Nội dung công việc	Số giờ thực tế

Chung cả nhóm	Tìm hiểu về đề tài và các dịch vụ sử dụng trong đề tài	4 giờ
Phan Nguyễn Quốc Hoàng	Nghiên cứu AWS Student Guide	1 giờ
	Thao tác trên console: Tạo S3, cấu hình Policy, tạo bảng DynamoDB (GSI, On-demand), gắn Tag (Thành viên còn lại theo dõi việc thực hiện thông qua Google Meet)	3 giờ
Phạm Trần Đức Lương	Thiết kế kiến trúc, lập sơ đồ kiến trúc hệ thống và thiết kế sơ bộ CSDL	2 giờ
	Tổng hợp viết báo cáo tuần 1	2 giờ

Tuần 2: Tổng thời gian: Phan Nguyễn Quốc Hoàng thực hiện 9.5 giờ, Phạm Trần Đức Lương thực hiện 9.5 giờ.

Thành viên	**Nội dung công việc**	**Số giờ thực tế**
Phan Nguyễn Quốc Hoàng	Thiết lập quyền thực thi LabRole. Cấu hình API Gateway (Resource /students, ánh xạ 4 HTTP Methods qua Lambda Proxy Integration)	3.0 giờ
	Cấu hình bảo mật CORS, thực hiện Deploy API lên môi trường dev và trích xuất Invoke URL	2.0 giờ
	Viết mã JavaScript tích hợp Fetch API vào index_test.html. Trực tiếp kiểm thử End-to-End luồng dữ liệu. Tổng hợp hình ảnh và định dạng báo cáo	4.5 giờ
Phạm Trần Đức Lương	Nghiên cứu boto3, khởi tạo và viết mã Python cho hàm CreateStudent và	3.5 giờ

	GetStudents (Bao gồm logic quét toàn bảng và truy vấn qua GSI major-index)	
	Viết mã Python cho UpdateStudent và DeleteStudent	2.0 giờ
	Thiết lập các Test Event (định dạng JSON) để kiểm thử độc lập 4 hàm trực tiếp trên AWS Lambda. Thực hiện Unit Test trực tiếp 4 hàm trên AWS Console. Điều chỉnh UpdateStudent và DeleteStudent để áp dụng ConditionExpression để xử lý ngoại lệ ConditionalCheckFailedException và các lỗi nghiệp vụ khác gặp phải. Tổng hợp hình ảnh và định dạng báo cáo	4.0 giờ

Tuần 3: Tổng thời gian: Phan Nguyễn Quốc Hoàng thực hiện 8.5 giờ, Phạm Trần Đức Lương thực hiện 8 giờ.

Thành viên	Nội dung công việc	Số giờ thực tế
Phan Nguyễn Quốc Hoàng	Tạo API Key, cấu hình Usage Plan và giới hạn Rate/Burst/Quota; liên kết Usage Plan với stage dev và API Key	3.0 giờ
	Cấu hình quyền ghi log cho API Gateway bằng LabRole, tạo CloudWatch Log Group, bật Access Logging và kiểm tra log/metrics. Tổng hợp hình ảnh và định dạng báo cáo	4.0 giờ

	Tải Frontend lên S3, kiểm thử E2E trên website và xuất dữ liệu Usage Plan	1.5 giờ
Phạm Trần Đức Lương	Cập nhật giao diện frontend/index.html, tích hợp API URL và API Key lấy từ AWS Console; gửi API Key trong header x-api-key khi gọi API. Tổng hợp hình ảnh và định dạng báo cáo	4.5 giờ
	Kiểm thử các chức năng trên website S3, xác nhận thao tác tạo, cập nhật và xóa sinh viên hoạt động bình thường	2.0 giờ
	Soạn định dạng JSON cho Access Log của API Gateway, xác định các thông tin request cần ghi nhận	1.5 giờ

Tuần 4: Tổng thời gian: Phan Nguyễn Quốc Hoàng thực hiện 9 giờ, Phạm Trần Đức Lương thực hiện 8.5 giờ.

Thành viên	**Nội dung công việc**	**Số giờ thực tế**
Phan Nguyễn Quốc Hoàng	Kiểm thử giao diện E2E trên Web S3: Test các thao tác thêm, sửa, xóa, lọc kèm hiển thị lỗi thông báo khi trùng ID, sai định dạng email, sai GPA	1.0 giờ
	Tìm hiểu và xuất file đặc tả OpenAPI 3 từ API Gateway	0.5 giờ
	Đọc, phân tích và trích xuất báo cáo Cost & Billing, CloudWatch Metrics. Viết báo cáo tuần 4	1.5 giờ

	Quay video phần cài đặt dự án và test E2E; viết báo cáo phần System Architecture, System Implementation, Security Configuration, Cost Analysis	6.0 giờ
Phạm Trần Đức Lương	Rà soát và viết lại logic xử lý lỗi cho 4 hàm Lambda (Create, Get, Update, Delete) trực tiếp trên console nhằm tối ưu mã nguồn và bẫy lỗi nghiệp vụ chặt chẽ	1.0 giờ
	Cập nhật file Frontend index.html bổ sung UI/UX cho tính năng tìm kiếm, lọc danh sách sinh viên qua API; xử lý logic hiển thị các bảng thông báo xác nhận và thông báo lỗi	0.5 giờ
	Tìm hiểu và xây dựng bộ test Postman Collection toàn diện	1 giờ
	Quay video demo phần test Postman và viết báo cáo phần Introduction, Testing and Monitoring, Problems Encountered and Lessons Learned, Team Contribution and AI Usage Declaration	6 giờ

8.2. Khai báo sử dụng AI

Tuần 1: Mô hình sử dụng: Gemini, phiên bản 3.1 Pro.

- Prompt: Tạo cho tôi một file JSON để cấu hình bảng DynamoDB tên là Students. Bảng cần có khóa chính, một GSI để truy vấn, thiết lập BillingMode tính phí tự động và có khai báo sẵn cấu trúc Tags cơ bản.

- AI tạo ra: AI đã cung cấp bộ khung JSON chuẩn cú pháp AWS DynamoDB, nhưng các thông số giả định (tên trường, chế độ thanh toán) chưa bám sát thực tế.
- SV đã sửa: Sinh viên đã chủ động tinh chỉnh cấu trúc JSON: Đổi tên trường thành student_id và major cho đúng thiết kế; thay đổi cấu hình BillingMode sang PROVISIONED với thông số RCU/WCU cụ thể; và cập nhật chính xác giá trị Tag của dự án theo mã số sinh viên.

  Tuần 2: Mô hình sử dụng: Gemini, phiên bản 3.1 Pro.

- Prompt: Tối ưu hóa mã nguồn AWS Lambda xử lý cập nhật dữ liệu để hỗ trợ cơ chế Dynamic Update, và xử lý lỗi ConditionalCheckFailedException (Mã HTTP 404) khi sinh viên không tồn tại trong DynamoDB.
- AI tạo ra: AI đã đề xuất đoạn mã nguồn Python sử dụng thư viện boto3 linh hoạt bằng vòng lặp để tạo các tham số UpdateExpression, ExpressionAttributeNames, và ExpressionAttributeValues. Đồng thời AI bổ sung ConditionExpression="attribute_exists(student_id)" và khối try-except bắt lỗi botocore.exceptions.ClientError.
- SV đã sửa: Tinh chỉnh thông báo lỗi trong JSON trả về thành tiếng Việt thân thiện với người dùng. Tự áp dụng cấu trúc ConditionExpression tương tự cho hàm DeleteStudent để hệ thống được chặt chẽ hơn trong luồng xóa dữ liệu.

  Tuần 3: Mô hình sử dụng: Gemini, phiên bản 3.1 Pro.

- Prompt: Phân tích nhu cầu giám sát một REST API xây dựng trên Amazon API Gateway. Đề xuất cấu trúc Access Log dạng JSON phù hợp, giải thích các nhóm thông tin nên ghi nhận để theo dõi request và hỗ trợ kiểm tra, gỡ lỗi. Có thể đề xuất thêm các trường hữu ích khác, nhưng cần lưu ý đến mức độ cần thiết và tránh ghi nhận dữ liệu nhạy cảm.
- AI tạo ra: AI phân tích các nhu cầu giám sát và đề xuất cấu trúc Access Log với nhiều thông tin về request, nguồn truy cập, thời điểm xử lý và kết quả phản hồi. AI cũng giải thích mục đích của từng nhóm trường và gợi ý các thông tin bổ sung có thể cân nhắc.
- SV đã sửa: Quá trình nhóm tinh chỉnh: Chọn các trường phù hợp với phạm vi dự án: requestId, ip, requestTime, httpMethod, routeKey, status, protocol và responseLength.

  Tuần 4: Không sử dụng AI.

TÀI LIỆU THAM KHẢO

1. Building Lambda functions with Python – AWS Documentation. Truy cập: https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html
2. Boto3 Documentation: DynamoDB Service Resource. Truy cập: https://docs.aws.amazon.com/boto3/latest/reference/services/dynamodb.html
3. Creating and using usage plans with API keys – Amazon API Gateway. Truy cập: https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-usage-plans.html
4. Postman Learning Center: Using environments and variables. Truy cập: https://learning.postman.com/docs/use/send-requests/variables/variables/
5. Hosting a static website using Amazon S3. Truy cập: https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html